Compress, Deploy, Adaptive Inference? Energy-Aware Visual Anomaly Detection on a Cortex-M Microcontroller

Ilka Bretschneider
ilka.bretschneider@studenti.unitn.it
University of Trento
Trento, Italy

Romain Noblet
romainmathis.noblet@studenti.unitn.it
University of Trento
Trento, Italy

ABSTRACT

Visual anomaly detection is attractive for low-cost quality inspection. Deploying it on battery-powered microcontrollers (MCUs) is constrained by tight memory, limited compute, and energy budgets. We build a complete pipeline that trains a compact convolutional autoencoder anomaly detector on the MVTec AD dataset, compresses it and deploys it as a fully-integer INT8 model on an Arduino Nano 33 BLE Sense Rev 2 (nRF52840, Cortex-M4F). We compress through quantization, structured pruning, and knowledge distillation and the combination of the three.

On real hardware the deployed models reach up to 0.946 image-level AUROC on the bottle category—within 0.02 of the floating-point software baseline—at 86 KB of SRAM and roughly 43 mJ per inference. We then study *energy-aware adaptive inference*: running simulations for switching among compressed models at runtime according to the available energy. Contrary to our initial hypothesis, runtime adaptation does *not* beat a carefully compressed static model for the given workload. Since the 50%-pruned (L1-Norm) model works at a robust operating point that clears the set quality floor of 0.86 at 39% lower energy than the most accurate model, it has the best accuracy–energy trade-off.

Our contribution is therefore the following: a measured, reproducible compression-to-deployment pipeline for MCU anomaly detection, and a simulation of energy-aware adaptation aiming to answer the question of when adaptation or a single static choice is optimal.

1 MOTIVATION

Automated visual inspection—detecting scratches, cracks, contamination, or counterfeit parts on a production line—can be performed by cloud-connected cameras or industrial PCs. By pushing this capability onto a self-contained, battery-powered microcontroller, privacy concerns and latency inference can be addressed. It would make it deployable anywhere: on a fast-moving production line (conveyor), inside a sealed device, or in a remote location without connectivity [5].

The problem this work attacks is making visual *anomaly detection*—learning what "normal" looks like and flagging deviations—run within the severe resource envelope of a commodity MCU. Technical challenges are limited computation, memory and processing power. State-of-the-art VAD methods employing computationally expensive operations become infeasible for edge deployment. Models detecting defects well do not necessarily *fit* on a microcontroller [5].

Additionally, edge studies often stop at simulated hardware instead of physically deploy. Since discrepancies exist between estimated and actual hardware performance, complete hardware deployment becomes crucial for evaluation embedded models [1].

A Cortex-M class device typically offers a few hundred kilobytes of SRAM, a single-digit-MHz-to-tens-of-MHz clock, no floating-point unit, and a finite energy budget.

Anomaly detection compounds the difficulty: it trains only on defect-free images, so the model must reconstruct or characterize normality precisely enough that anomalies stand out [2]. This has to be small and cheap enough to run on a memory and computation constrained device. We address this problem while also understanding how to spend a limited energy budget wisely, once a carefully compressed model is deployed.

2 LIMITATIONS OF THE STATE OF THE ART

Reconstruction- and feature-based anomaly detectors such as autoencoders, PaDiM [4], and EfficientAD [2] achieve strong accuracy on benchmarks like MVTec AD [3], but are designed for GPU or desktop-CPU inference.

PaDiM, for example, requires backbone weights and uses pretrained parameters. By storing per-patch Gaussian statistics from a deep backbone, it reaches > 0.99 AUROC on some categories. Yet its memory footprint and reliance on full-precision feature extraction place it far outside an MCU budget ($\sim$170 MB of backbone weights and Gaussian parameters) [4].

Recent edge-oriented variants investigate compression techniques to reduce parameter size, computational complexity and memory consumption [5]. Studies catalogue those

techniques available to thereby make quantized neural networks deployable for MCUs [1]. Yet, two gaps remain.

First, much reported "edge" work stops at a simulated or desktop-emulated quantized model and does not measure latency, memory, and energy on the target device [5]. Still, toolchain realities like operator support, activation memory and kernel acceleration dominate hardware performance, which creates discrepancies between estimated and actual performance [1]. Second, most literature investigates *static* compression, choosing one model throughout operation [5], [2]. We found no studies that evaluate runtime switching between multiple anomaly detection models to adapt computational cost to battery state or energy availability. Consequently, it remains unclear whether the additional complexity of *runtime energy-aware adaptation* is justified by improvements in energy efficiency or battery lifetime. This represents an open research question and has been identified as a promising direction for future work [1].

Our work closes the first gap with end-to-end on-device measurement and directly interrogates the second.

3 KEY INSIGHTS

Three insights drive this paper.

(1) On our MCU, activation memory–not parameter count—decide deployability. The fix *improved* accuracy.

Initially the model was fed training images with a resolution of 128×128. The "tensor arena" is dominated by activations maps and stored in SRAM ("256 KB"). Even fully quantized to INT8 the model required ~576 KB of activation arena, more than twice the device's SRAM, making it undeployable.

The biggest lever to address this problem, is reducing the input resolution to 64×64. This cut the arena into budget and unexpectedly *raised* the AUROC-score (bottle $0.93 \rightarrow 0.97$, hazelnut $0.83 \rightarrow 0.87$).

Since an autoencoder is trained to reconstruct images, at higher resolutions it also reconstructs fine-grained texture, sensor noise etc. Therefore the tighter bottleneck prevents the autoencoder from reconstructing defects, increasing their reconstruction error and hence their separability. Small texture variations are smoothed and the model focuses on larger structural features.

(2) The hardware constraints of our MCU showed the importance of architecture choices, that a software-only study would not surface. Choosing the right operator support *improved* latency.

Initially, the decoder used transposed convolutions for usampling. By replacing the decoder's transpose convolutions with nearest-neighbour upsampling followed by ordinary convolutions, on-device latency was reduced 5.6× (from ~11 s to ~2 s) at equal accuracy.

Some operations have highly optimized implementations written for ARM Cortex-M processors. Redesigning the network with "Conv2D" uses the CMSIS-NN library, accelerating specific operations. Since both architectures can learn the same mapping, accuracy remains unchanged.

(3) When one compressed model is both accurate enough and cheap enough, runtime energy adaptation provides no benefit—a single static choice is optimal.

Intuitively, we expected adaptive model-switching to dominate. Instead, across fixed-battery and energy-harvesting scenarios and both categories, statically selecting the cheapest model that clears the quality floor (of 0.86) beat every adaptive policy.

The cause is carefully applied compression: the 50%-pruned model retains near-baseline accuracy (0.946 on bottle, 0.884 on hazelnut) at 39% lower energy than the most accurate model. This creates a robust operating point with a positive accuracy–energy trade-off.

Adaptation could only be justified under a wide accuracy–energy spread or a dynamically changing quality floor.

4 MAIN ARTIFACTS

The central artifact is a four-stage, measured compression-to-deployment pipeline, together with the on-device firmware and the analysis that evaluates energy-aware adaptation.

Stage 1: Detect — This stage implements a baseline visual anomaly detector. A compact convolutional autoencoder is trained on defect-free images from the MVTec AD dataset. During inference, anomalous regions are identified through the reconstruction error.

The images fed to the autoencoder are reduced in size to 64×64, center cropped and augmented to create robustness against small changes (random horizontal flip/ small random rotation). The loss function is put together by the Mean Squared Error (MSE) that directly compares pixel values and the Structural Similarity Index Measure (SSIM) that measures structural similarities, which is where the defects are.

$$\mathcal{L}_{\text{recon}}(\hat{x}, x) = (1 - \lambda)\text{MSE}(\hat{x}, x) + \lambda\,(1 - \text{SSIM}(\hat{x}, x))\,, \quad (1)$$

The anomaly scoring follows a *frozen protocol* reused identically at every stage to ensure AUROC comparisons are valid. The anomaly score is computed based on the per-pixel reconstruction error between the input image and its reconstruction. Before calculating the final anomaly score, a border crop is applied to remove unreliable edge regions. Convolutional networks often produce less reliable reconstructions near image boundaries because filters have incomplete spatial context there. Removing these edge areas helps prevent such artifacts from negatively influencing the results. After cropping, a 5×5 average-pooling operation is applied. Finally, a spatial mean operation is used to convert the pixel-level

anomaly map into a single anomaly score per image. All these operations are used to obtain usable separability.

We evaluate on the `bottle` and `hazelnut` MVTec categories. Texture-dominated categories (e.g. carpet, grid) and categories with subtle, low-contrast defects (e.g. metal_nut, pill) were excluded as documented autoencoder failure cases. Since AUROC-scores where too low, the reconstruction-based detector could not reliably flag defects.

Stage 2: Compress — This stage takes the FP32 baseline models from stage 1 and applies three different compression families. It then characterizes the accuracy/size/compute trade-off.

A sweep over post-training INT8 quantization, L1- and Taylor-criterion structured filter pruning (30%/50%), knowledge distillation, and their combinations is characterized by accuracy-vs-cost Pareto fronts per category.

Since GPUs do not execute INT8 inference in the same way as the target MCUs, PyTorch quantization is used to estimate the accuracy impact of INT8 by approximating both weight and activation quantization. Consequently, Stage 2 reports the theoretical INT8 model size, while real memory usage is measured during Stage 3 on the target hardware.

Through structured filter pruning, entire filters (output channels) are removed to shrink the architecture. When using the L1-Norm criterion, each filter is pruned based on an importance score built on the magnitude of its weights. When using the Taylor-Norm, the loss if the filter was removed is approximated by first-order gradient information. It is pruned by taking the gradient of the loss with respect to that weight into account. The sweep retrains each pruned candidate briefly to recover accuracy.

Knowledge Distillation trains a smaller student Autoencoder that matches the teacher's reconstruction and the ground-truth input. This allows the student to learn the richer representations captured by the larger model instead of only relying on information in input pixels. The teacher model uses 32 base channels and 32 latent channels, while the student is reduced to 16 base channels and 16 latent channels, resulting in a more compact architecture with fewer feature maps and a tighter latent representation. Training minimizes a weighted loss of the following form (2):

$$\mathcal{L} = \alpha\, \mathcal{L}_{\mathrm{SI}}(\mathbf{s}, \mathbf{x}) + (1 - \alpha)\, \mathcal{L}_{\mathrm{ST}}(\mathbf{s}, \mathbf{t}), \tag{2}$$

Every variant is scored with the identical frozen protocol so that points are directly comparable. Figure 1 shows the resulting accuracy–size pareto fronts. Pruning consistently lies on or above the distillation points while INT8 quantization moves a model leftward (smaller) with negligible vertical (accuracy) cost.

Stage 3: Deploy — This stage takes the compressed models, converts them to TFLite INT8 models and deploys them on the Arduino Nano 33 BLE Sense Rev 2.

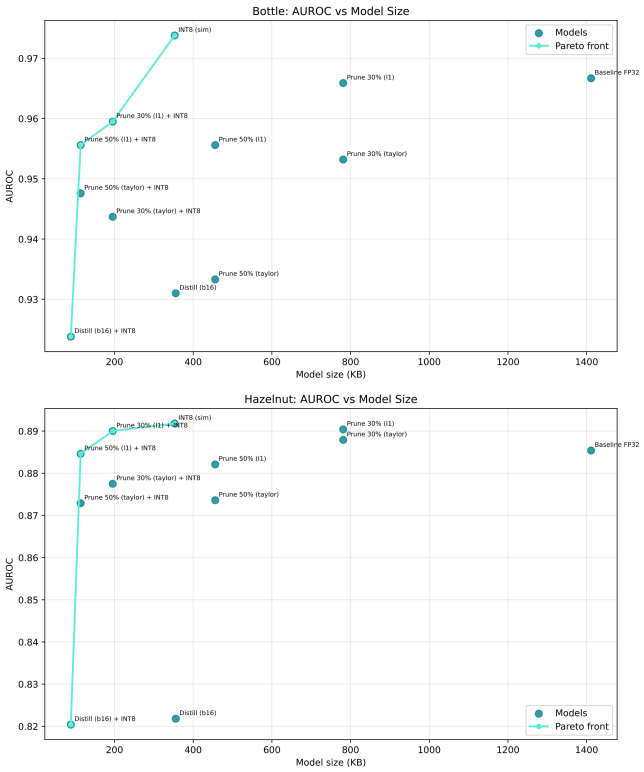

Figure 1: Accuracy–size Pareto fronts for the compression sweep. Pruning dominates distillation on both categories. INT8 quantization is nearly accuracy-free. The 50%-pruned model is the robust low-energy operating point that later makes runtime adaptation unnecessary.

It then measures memory footprint and real latency to approximate energy per inference.

Each selected model is converted from PyTorch → LiteRT → to a static full-integer INT8 (weights *and* activations quantized via a calibrated representative dataset) model. It is then compiled into firmware for the Arduino Nano 33 BLE Sense Rev 2.

The deployment set consists of three representative variants selected from the Stage 2 Pareto fronts. The models span the accuracy–efficiency trade-off:

- the 30% L1-pruned model as the highest-accuracy deployable point
- the 50% L1-pruned model as a near-baseline alternative with approximately half the MACs
- the distilled model with 16 base and latent channels as the smallest, lowest-cost configuration

The full scoring protocol is reimplemented in C. By streaming the entire test set over the serial port, we measure:

- on-device latency (`millis` around `Invoke`)

- SRAM arena (`arena_used_bytes`)
- flash
- AUROC

Per-inference energy is estimated from measured latency and the nRF52840 datasheet active current (6.3 mA at 3.3 V, ~20.8 mW). Table 1 summarizes the measured deployment characteristics. Figure 2 shows the accuracy–energy trade-off with on-device measurements from Stage 3.

Table 1: Measured on-device characteristics (Arduino Nano 33 BLE Sense Rev 2, nRF52840 @ 64 MHz). Energy from measured latency and datasheet power (20.8 mW). AUROC is measured on-device over the full test set with the frozen scoring protocol.

Category	Model	Lat. (ms)	Energy (mJ)	AUROC
bottle	Prune 30% L1	3355	69.7	0.960
bottle	Prune 50% L1	2059	42.8	0.946
bottle	Distill (b16)	1992	41.4	0.937
hazelnut	Prune 30% L1	3356	69.8	0.890
hazelnut	Prune 50% L1	2060	42.8	0.884
hazelnut	Distill (b16)	1992	41.4	0.823

Stage 4: Adapt — This stage implements simulations of a depleting battery and a bursting energy harvesting buffer. For evaluation, it implements two adaptive strategies and compares those to static deployment by total inferences served.

The goal is to hold a ladder of compressed models and switch at runtime based on remaining energy. When energy is plentiful, the MCU should run an accurate (expensive) model while as the batter depletes, the MCU should drop to a cheaper model, trading accuracy for survival. The simulation is entirely parameterized by the *measured* per-model energy and on-device accuracy.

The threshold-based policy partitions the battery state into fixed charge bands, selecting the 30% L1-pruned model above 66%, the 50% L1-pruned model between 33% and 66%, and the distilled model below 33%, thereby implementing a predetermined accuracy-versus-energy schedule.

In contrast, the utility-based policy makes a decision at every inference by estimating the remaining number of inferences that could be executed using the cheapest model and selecting the most accurate model that still guarantees a predefined survival budget of 50 future inferences. This formulates runtime model selection as a constrained optimization problem that maximizes inference accuracy subject to satisfying the minimum remaining-lifetime requirement.

Both adaptive strategies are compared against fixed-model baselines using an accuracy-floor objective.

5 KEY RESULTS AND CONTRIBUTIONS

Headline results. The deployed 30% L1-pruned model on the full bottle test set achieves 0.946 on-device AUROC within ~0.02 of the 0.967 software baseline. For the category hazelnut, the model achieves 0.89 on-device AUROC within ~0.005 of the 0.885 software baseline. This confirms that full-integer quantization and the C-side scoring protocol preserve detection quality on real hardware. The small fluctuation is inside the test-set noise.

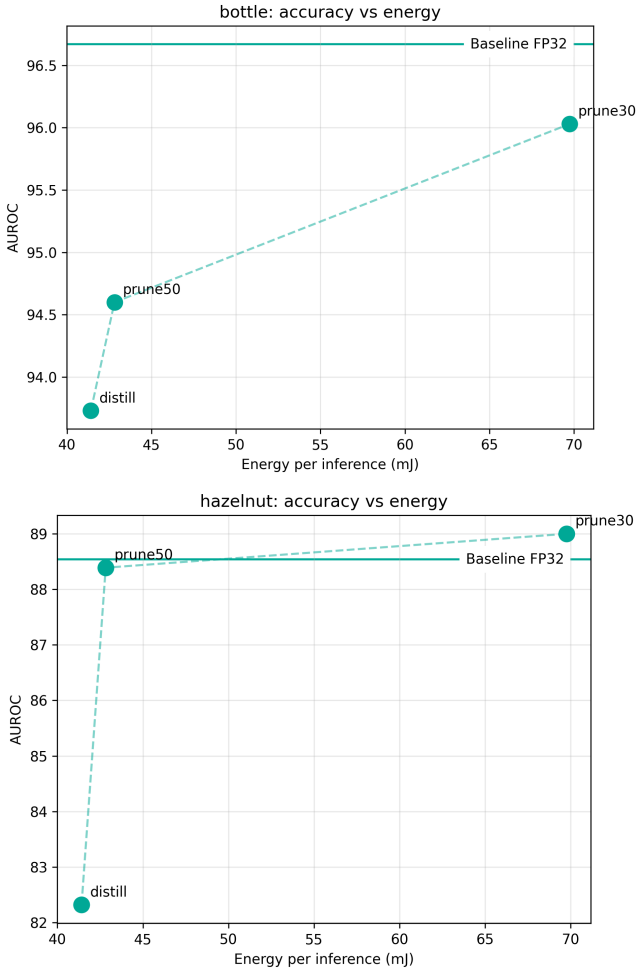

Figure 2: Accuracy–Energy trade-off with on-device measurements. The deployed 30% L1-pruned model on the full bottle test set achieves 0.946 on-device AUROC within ~0.02 of the 0.967 software baseline. For the category hazelnut, the model achieves 0.89 on-device AUROC within ~0.005 of the 0.885 software baseline.

In the energy study under a depleting battery and an accuracy floor of 0.86, the evaluation of:

- total possible inferences

- $inferences \geq AUROC$ 0.86
- mean AUROC

gives a clear result.

For Bottle, all strategies consistently preserve high-quality outputs, achieving full coverage above the threshold. The always-50%-pruned model serves 111 above-floor inferences, which is 42 more than the most accurate model always-30%-pruned and more than any adaptive policy as displayed in figure 3. Still it maintains a mean AUROC score of 94.6 which is within ~2 of the Baseline FP32 model.

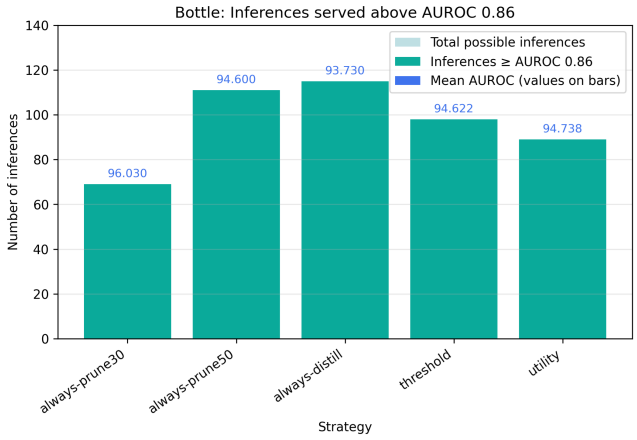

Figure 3: Inferences served for Bottle. All strategies consistently preserve high-quality outputs, achieving full coverage above the threshold.

For Hazelnut, distillation reduces quality below the required floor, while pruning-based strategies maintain higher AUROC compliance. The always-50%-pruned model also serves 111 above-floor inferences, which is 42 more than the most accurate model always-30%-pruned and more than any adaptive policy as displayed in figure 4.

This shows that the always-50%-pruned model has a robust, low-energy, floor-clearing operating point.

Adaptation did not win under the simulation of a bursty harvesting buffer either, for the same reason.

As shown in figure 5 for Bottle, the always-30%-pruned model provides the highest mean AUROC (96.03) but serves the fewest inferences due to its higher energy cost similar to the battery simulation. In contrast, the always-50%-pruned and distilled models significantly increase the number of served inferences. The adaptive policy provides an intermediate operating point, maintaining a mean AUROC of 94.76. Still it is not beneficial over the always-50%-pruned model.

For Hazelnut, the accuracy–energy trade-off is more restrictive. The distilled model achieves zero inferences above the required accuracy floor. The pruning-based models provide the most robust operating points similar to the battery

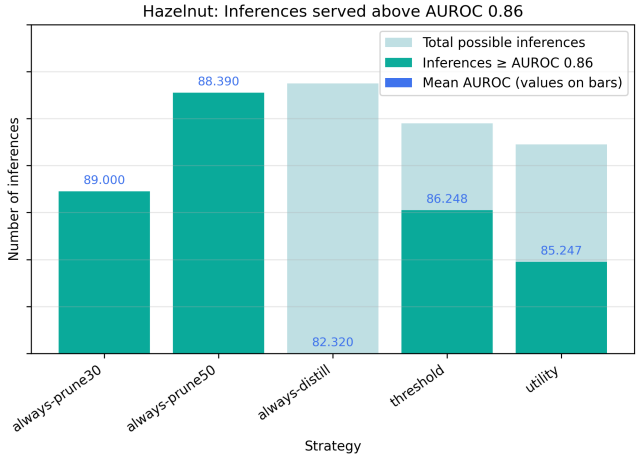

Figure 4: Inferences served for Hazelnut. Distillation reduces quality below the required floor, while pruning-based strategies maintain higher AUROC compliance. Adaptive policies do not improve on the best static choice.

simulation. The adaptive policy successfully avoids the quality loss of the distilled model, but does not add a benefit over the always-50%-pruned model.

Why adaptation does not help here. The energy ladder spans only 41–70 mJ while accuracy spans only a few points, so the cheapest floor-clearing model maximizes the number of acceptable inferences delivered from a fixed charge. Figure 5 makes this concrete: the static 50%-pruned policy serves the most above-floor inferences, while always running the most accurate model exhausts the battery far sooner and any floor-unaware adaptive policy wastes energy on accuracy it does not need (or drops below the floor entirely). We also tested a bursty energy-harvesting regime with a small supercapacitor buffer (Figure 6), the setting most favourable to adaptation: even there, statically selecting the cheapest floor-clearing model matched the best adaptive policy, because that model is sustainable on the mean harvested power. Adaptation would only pay off if the cheapest acceptable model were itself unaffordable during lulls, or if the accuracy floor changed at runtime—conditions absent from this workload but which our analysis identifies precisely.

Supporting findings. (i) Reducing input resolution from 128 × 128 to 64 × 64 was *required* to fit the 256 KB SRAM budget and improved AUROC by ~4 points in both categories. (ii) Replacing transpose-convolution decoders with upsample+conv reduced latency 5.6× by enabling CMSIS-NN acceleration. (iii) In compression, INT8 was nearly accuracy-free, pruning Pareto-dominated distillation, and L1 pruning matched or beat the Taylor criterion at aggressive ratios. (iv)

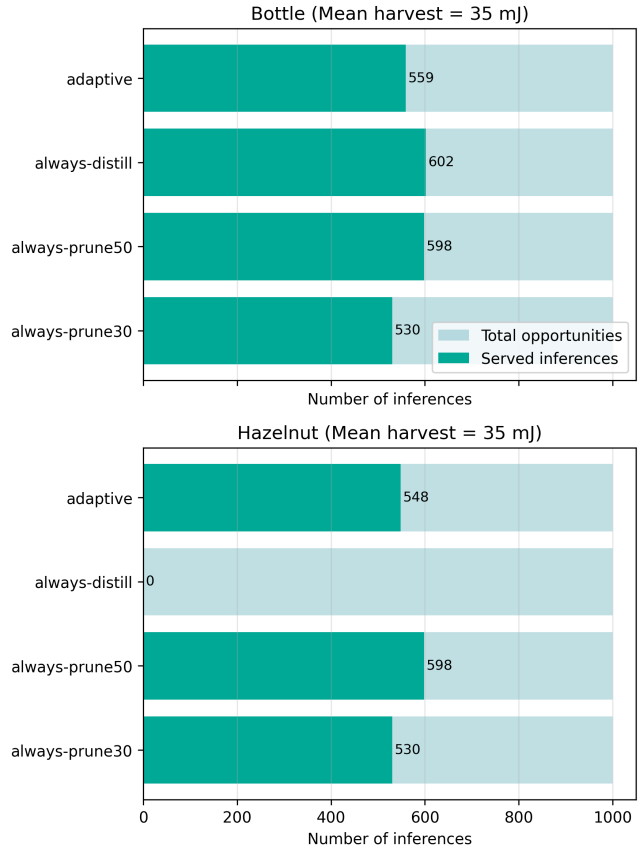

Figure 5: Inferences served for Bottle and Hazelnut on a bursty harvesting buffer.

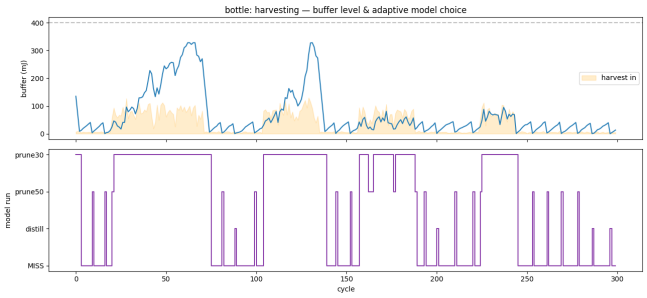

Figure 6: Energy-harvesting scenario. The adaptive policy tracks the buffer level—running costlier models at peaks, cheaper ones during lulls—yet does not exceed a well-chosen static policy, because the 50%-pruned model is sustainable on the mean harvested power.

The two categories told opposite stories that persisted end-to-end: `bottle` defects are easy (every compressed model clears any reasonable floor), whereas `hazelnut` exposes the real trade-offs, making it the category where adaptation *could* matter—and still did not.

Contributions.

- A complete, reproducible pipeline that takes an autoencoder anomaly detector from training through compression to a *measured* full-integer INT8 deployment on a commodity Cortex-M4F MCU, with on-device AUROC matching the software baseline to within 0.02.
- Two hardware-driven design lessons surfaced only by on-device measurement: input resolution is bounded by activation memory (and lowering it improved accuracy), and transpose-convolution decoders are a latency bottleneck remedied by upsample+conv (5.6× speedup).
- A measurement-grounded evaluation of energy-aware adaptive inference showing that, for a robust compressed model, a single static choice is optimal under both fixed-battery and harvesting conditions, together with a precise statement of the conditions (wide accuracy–energy spread, dynamic floor, or extreme scarcity) under which adaptation would instead be warranted.

Advantages over prior work. Unlike studies that stop at emulated quantization, every cost here is measured on the target device, exposing the memory, operator, and kernel realities that decide deployability. And where prior work assumes energy-aware adaptation is beneficial, we test it directly with real numbers and delineate when it is not—turning a negative result into an actionable deployment guideline: select the cheapest compressed model that clears your quality floor, and reserve runtime adaptation for genuinely variable operating conditions. This connects directly to ongoing interest in on-device counterfeit and defect detection, where untethered, energy-limited inspection is the practical goal.

6 CONCLUSION

We built and measured a complete path from an autoencoder anomaly detector to a fully-integer INT8 deployment on a commodity Cortex-M4F microcontroller, reaching on-device accuracy within 0.02 of the software baseline. Along the way, two hardware constraints—activation-memory limits and the absence of fast transpose-convolution kernels—drove architecture changes (lower input resolution, upsample+conv decoders) that improved both deployability and, in the first case, accuracy. Finally, we evaluated energy-aware adaptive inference and found that, for a model family with a robust mid-compression operating point, a single static model selection is optimal under both fixed-battery and energy-harvesting

conditions. The practical takeaway for edge anomaly detection is to compress aggressively, deploy and measure on the real device, and adapt at runtime only when the energy supply or quality requirement genuinely varies. Future work includes measuring per-inference energy with a current probe to replace the datasheet estimate, implementing the selection policy in firmware as a live demonstration, and extending the study to the dynamic-floor and extreme-scarcity regimes where adaptation is predicted to help.

REFERENCES

[1] Hamza A. Abushahla, Dara Varam, Ariel Justine N. Panopio, and Mohamed I. AlHajri. Neural network quantization for microcontrollers: A comprehensive survey of methods, platforms, and applications. *arXiv preprint arXiv:2508.15008*, 2025.

[2] Kilian Batzner, Lars Heckler, and Rebecca König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 128–138, 2024.

[3] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9592–9600, 2019.

[4] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. PaDiM: A patch distribution modeling framework for anomaly detection and localization. In *International Conference on Pattern Recognition (ICPR) Workshops*, pages 475–489, 2021.

[5] Arianna Stropeni, Fabrizio Genilotti, Francesco Borsatti, Manuel Barusco, Davide Dalle Pezze, and Gian Antonio Susto. Efficient visual anomaly detection at the edge: Enabling real-time industrial inspection on resource-constrained devices. *arXiv preprint arXiv:2603.20288*, 2026.